



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

System wspomagający inteligentne składanie zestawów komputerowych

Piotr KUPCZYK

Nr albumu: 305198

Kierunek: Teleinformatyka S1

Specjalność: Brak

PROWADZĄCY PRACĘ

dr hab. inż. Bożena Małysiak-Mrozek

KATEDRA Systemów Rozproszonych i Urządzeń Informatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2025

Tytuł pracy

System wspomagający inteligentne składanie zestawów komputerowych

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

A system supporting intelligent assembly of computer sets

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

Spis treści

1	Wstęp	1
1.1	Wprowadzenie w problem	2
1.2	Osadzenie problemu w dziedzinie	2
1.3	Cel pracy	2
1.4	Zakres pracy	3
1.5	Zwięzła charakterystyka rozdziałów	4
1.6	Wkład autora	5
2	Analiza problemu oraz przegląd dostępnych rozwiązań	7
2.1	Sformułowanie problemu	8
2.1.1	Istota kompatybilności	8
2.1.2	Niska dostępność narzędzi do weryfikacji kompatybilności	8
2.2	Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku	8
2.2.1	Morele.net	8
2.2.2	X-kom.pl	9
2.3	Porównanie istniejących rozwiązań z rozwiązaniem inżynierskim	9
3	Wymagania i narzędzia	11
3.1	Wymagania funkcjonalne	12
3.2	Wymagania нефункционалне	13
3.3	Przypadki użycia	13
3.4	Narzędzia i technologie	14
3.4.1	Frontend	14
3.4.2	Backend	14
3.4.3	Modelowanie i narzędzia wspomagające	14
3.5	Metodyka pracy	15
4	Specyfikacja wewnętrzna	17
4.1	Przedstawienie idei systemu	18
4.1.1	Cel powstania aplikacji	18

4.1.2	Innowacyjność systemu	19
4.2	Architektura systemu	20
4.2.1	Schemat ideowy	20
4.2.2	Frontend	21
4.2.3	Backend	22
4.2.4	Baza danych	23
4.2.5	Infrastruktura chmurowa	24
4.3	Baza danych	28
4.3.1	Model danych - ERD	28
4.3.2	Opis najważniejszych encji	30
4.4	Komponenty i moduły systemu	42
4.4.1	Moduł kompatybilności podzespołów	42
4.4.2	Moduł logiki rozmytej	42
4.4.3	Moduł zarządzania koszykiem i zamówieniami	42
4.5	Najważniejsze algorytmy	42
4.6	Diagramy UML	42
4.7	Szczegóły implementacji wybranych elementów	42
5	Weryfikacja i walidacja	43
5.1	Cel weryfikacji systemu	44
5.2	Metody testowania	44
5.3	Organizacja eksperymentów	44
5.4	Przypadki testowe	44
5.5	Wyniki testów	44
5.6	Wykryte i usunięte błędy	45
5.7	Ocena poprawności działania systemu	45
6	Podsumowanie i wnioski	47
6.1	Realizacja celów pracy	47
6.2	Ocena otrzymanych rezultatów	47
6.3	Trudności napotkane podczas realizacji	47
6.4	Możliwe kierunki dalszego rozwoju	47
6.5	Wnioski końcowe	47
	Bibliografia	49
	Spis skrótów i symboli	53
	Źródła	55
	Lista dodatkowych plików, uzupełniających tekst pracy	57

Spis rysunków	59
Spis tabel	61

Rozdział 1

Wstęp

1.1 Wprowadzenie w problem

Sklepy komputerowe zazwyczaj posiadają swój własny sklep internetowy, oferujący możliwość zakupów części indywidualnie, jak i gotowych zestawów PC. Odpowiedni dobór komponentów jest najistotniejszą częścią procesu projektowania własnego stanowiska, a pomimo tego najwięksi sprzedawcy nie oferują możliwości sprawdzania kompatybilności wybranych produktów. Dlatego projekt ma na celu stworzenie prostej i intuicyjnej strony internetowej, która zaoferuje przystępną formę weryfikacji poprawności wyboru komponentów z katalogu oraz finalnie dodanie kompatybilnego zestawu do koszyka użytkownika.

1.2 Osadzenie problemu w dziedzinie

Projekt osadzony jest w dziedzinie inżynierii oprogramowania, projektowania baz danych oraz systemów e-commerce. Praca obejmuje zagadnienia relacyjności baz danych, budowy warstwy komunikacyjnej backendu, implementacji logiki aplikacji pod kątem biznesowym oraz wdrożenie interfejsu frontendowego.

1.3 Cel pracy

Celem pracy jest zaprojektowanie i zaimplementowanie aplikacji webowej, która będzie pełnić funkcję inteligentnego kreatora zestawów komputerowych. System będzie dawać użytkownikowi możliwość wybrania dowolnych produktów z katalogu, sprawdzenie czy aktualne zamówienie jest wzajemnie kompatybilne, a gdy weryfikacja zakończy się pomyślnie, umożliwi realizację zamówienia.

Idące za tym cele to:

- stworzenie relacyjnej bazy danych, odpowiadającej potrzebom sklepu internetowego,
- zaprojektowanie i implementacja backendu, odpowiedzialnego za komunikację i logikę aplikacji,
- wdrożenie przejrzystego frontendu, będącego interfejsem klienta,
- stworzenie grupy zasobów w środowisku chmurowym Azure (baza danych jako storage, aplikacja jako kontener).

1.4 Zakres pracy

Zakres pracy nad projektem obejmuje:

- projekt relacyjnej bazy danych w Oracle SQL Developer.
- utworzenie testowego serwera PostgreSQL, wprowadzenie gotowych danych o produktach do bazy danych,
- stworzenie użytkowników oraz nadanie odpowiednich uprawnień każdemu z nich,
- implementację backendu w języku Python, implementacja API z wykorzystaniem frameworka FastAPI, umożliwiając integrację z bazą danych w celu zarządzania produktami i zamówieniami,
- projekt interfejsu użytkownika, stworzenie frontendu używając HTML, CSS oraz JavaScript,
- projekt kreatora zestawów komputerowych, zaimplementowanego z uwzględnieniem logik rozmytej, używając Reacta, Typescript, Node.js,
- migrację gotowej aplikacji z lokalnego serwera testowego PostgreSQL do produkcyjnej chmury Azure, utworzenie grupy zasobów.

1.5 Zwięzła charakterystyka rozdziałów

- **Rozdział 1** - zawiera wprowadzenie do tematyki pracy, przedstawia problem nieintuicyjnego doboru kompatybilnych podzespołów komputerowych, definiuje cel i zakres pracy oraz opisuje autorski wkład w projekt.
- **Rozdział 2** - obejmuje analizę tematu, w tym istotę kompatybilności sprzętowej, przegląd istniejących rozwiązań wykorzystywanych przez popularne sklepy komputerowe oraz porównanie ich funkcjonalności z zaprojektowanym systemem. Przybliża problematykę całej pracy.
- **Rozdział 3** - przedstawia wymagania funkcjonalne i нефункционалне systemu, przypadki użycia, zastosowane narzędzia i technologie oraz metodykę pracy przyjętą podczas implementacji. Przedstawia kompletną listę dobranych technologii i rozwiązań.
- **Rozdział 4** - stanowi część projektową, zawierającą architekturę systemu, opisanie każdego filaru aplikacji webowej, model danych wraz z diagramem ERD, opis struktury bazy danych, szczegółową analizę encji oraz omówienie kluczowych modułów, takich jak moduł weryfikacji kompatybilności i logiki rozmytej.
- **Rozdział 5** - prezentuje proces weryfikacji i walidacji systemu, zawierając opis sposobu testowania, organizacji eksperymentów, przypadków testowych oraz analizę wykrytych błędów i wyników testów. Analizuje skuteczność działania oprogramowania, porównując otrzymywane wyniki z założeniami.
- **Rozdział 6** - zawiera podsumowanie uzyskanych rezultatów, ocenę stopnia realizacji celów pracy oraz wskazuje możliwe kierunki dalszego rozwoju systemu. Podsumowuje i ocenia skuteczność podjętych działań, analizuje koszty poniesione podczas realizacji, szczegółowo ocenia dobór technologii.

1.6 Wkład autora

Autor samodzielnie opracował wszystkie elementy projektu, zaczynając od zaplanowania funkcjonalności oraz wyboru technologii. Następnym krokiem było zaprojektowanie relacyjnej bazy danych w Oracle SQL Developer, gdzie utworzył on testowy serwer PostgreSQL wraz z użytkownikami oraz nadał odpowiednie uprawnienia każdemu z nich. Przy użyciu frameworku FastAPI przygotowany został backend w języku Python, odpowiadający za komunikację z bazą danych oraz za logikę całej aplikacji, umożliwiając weryfikację kompatybilności zestawów komputerowych. Autor utworzył interaktywny frontend, który umożliwia graficzne korzystanie z aplikacji, zapewniając łatwość użytkowania zarówno dla doświadczonych, jak i początkujących inżynierów sprzętu, poprzez stosowanie logiki rozmytej. Korzystając z akademickiej subskrypcji Microsoft Azure autor używając otrzymanych kredytów samodzielnie realizuje hosting, administrację, testowanie CI/CD, konteneryzację oraz wirtualizację aplikacji webowej, czyniąc ją w pełni używalną w publicznej sieci internet. Dokumentacja przygotowana do projektu, dokument inżynierski oraz wszystkie opisy dostępne w projekcie zostały samodzielnie, szczegółowo opracowane w ramach projektu inżynierskiego "System wspomagający inteligentne składanie zestawów komputerowych".

Rozdział 2

Analiza problemu oraz przegląd dostępnych rozwiązań

2.1 Sformułowanie problemu

2.1.1 Istota kompatybilności

Na rynku znajduje się szeroka gama komponentów komputerowych, która nieustannie poszerza się o nowe produkty. Różnice pomiędzy poszczególnymi modelami dotyczą chociażby wymiarów fizycznych, dostępnych złączy, technologii socketów, minimalnej mocy zasilacza, aż do wymogów software'owych. Odpowiedni dobór części jest pierwszym krokiem do realizacji poprawnie funkcjonującego stanowiska komputerowego, gdyż dowolna pomyłka może prowadzić do braku możliwości fizycznego montażu lub niewłaściwej i nieciągłej pracy urządzenia. Nawet minimalne niezgodności mogą być przyczyną poważnej awarii, która może przyczynić się do strat finansowych, spowodowanych utratą danych, przerwaniem trwałości usług lub całkowitym uszkodzeniem sprzętu.

2.1.2 Niska dostępność narzędzi do weryfikacji kompatybilności

Aktualnie na polskim rynku sklepów komputerowych brakuje narzędzi umożliwiających użytkownikowi samodzielną i automatyczną weryfikację kompatybilności wybranych podzespołów. Większość sprzedawców oferuje wyłącznie gotowe zestawy komputerowe, które zapewniają pełną funkcjonalność, jednak często nie umożliwiają ich łatwej rozbudowy ani wymiany poszczególnych elementów na nowsze modele. W efekcie zaprojektowanie stanowiska w pełni odpowiadającego potrzebom użytkownika staje się utrudnione i może wymagać konsultacji ze specjalistą, co generuje dodatkowe koszty oraz wydłuża czas realizacji zamówienia.

2.2 Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku

2.2.1 Morele.net

Spośród przeanalizowanych platform sprzedażowych polskich sprzedawców, pod względem technicznym najlepiej zaprojektowanym rozwiązaniem okazał się konfigurator zestawów PC dostępny w sklepie Morele[2]. Narzędzie oferuje intuicyjny interfejs oraz możliwość wyboru szerokiej gamy podzespołów. Kluczowym ograniczeniem jest jednak brak automatycznej weryfikacji kompatybilności, gdzie w przypadku błędnie skonfigurowanego zestawu jedynie pracownik sklepu może ewentualnie skontaktować się z użytkownikiem. W praktyce znacząco opóźnia to proces realizacji zamówienia i niesie ryzyko zakupu podzespołów, które nie będą ze sobą wzajemnie kompatybilne. Dodatkowo konfigurator nie wykorzystuje logiki rozmytej przy sugerowaniu komponentów.

2.2.2 X-kom.pl

Drugim pod względem funkcjonalności rozwiązaniem jest konfigurator oraz oferta gotowych zestawów sklepu X-kom[4]. Platforma udostępnia czytelny interfejs oraz rekomendowane zestawy komputerowe, które zazwyczaj umożliwiają późniejszą rozbudowę. W porównaniu do Morele[2], użytkownik ma tutaj jeszcze mniejszą możliwość samodzielnego projektowania konfiguracji, ponieważ wybór ogranicza się do kilku lub kilkunastu proponowanych zestawów. Brak szczegółowej analizy potrzeb użytkownika oraz ograniczona liczba wariantów utrudniają złożenie w pełni dostosowanego zamówienia. Podobnie jak w przypadku Morele, sklep nie wykorzystuje logiki rozmytej w mechanizmach komponowania zestawów.

2.3 Porównanie istniejących rozwiązań z rozwiązaniem inżynierskim

Pierwszą i najważniejszą funkcjonalnością wyróżniającą opracowane rozwiązanie jest zastosowanie rozszerzonych mechanizmów weryfikacji kompatybilności podzespołów, których brakuje w analizowanych sklepach internetowych. System wykorzystuje nie tylko klasyczne metody porównywania parametrów technicznych, lecz również elementy logiki rozmytej pozwalającej na bardziej elastyczną ocenę dopasowania podzespołów do oczekiwań użytkownika.

Aplikacja dokonuje szczegółowej analizy technicznej wszystkich wybranych komponentów, co przekłada się na sprawdzanie i weryfikowanie:

- zgodności procesora i płyty głównej pod względem zastosowanego gniazda socket'u,
- typu pamięci RAM oraz jej zgodności ze standardem płyty głównej,
- maksymalnej obsługiwanej częstotliwość i pojemność modułów pamięci,
- parametrów energetycznych, takich jak zapotrzebowanie na moc (TDP) oraz wydajności i maksymalnej dostarczanej mocy zasilacza,
- wymiarów i formatów kart graficznych oraz obudowy,
- zgodności chłodzenia procesora z wybranym gniazdem i limitem wysokości w obudowie.

W odróżnieniu od popularnych serwisów takich jak Morele.net[2] czy X-kom[4], które nie oferują pełnej automatycznej weryfikacji kompatybilności, system analizuje wszystkie istotne zależności pomiędzy podzespołami. Dzięki temu użytkownik nie jest narażony na ryzyko zakupu części wzajemnie niekompatybilnych, a proces konfiguracji zestawu przebiega szybciej, sprawniej i bez konieczności konsultacji ze specjalistą. Zastosowanie logiki rozmytej pozwala ponadto na ocenę zestawów zarazem pod kątem poprawności technicznej, lecz także ich dopasowania do ogólnych kategorii takich jak budżet, wydajność czy energooszczędność.

Rozdział 3

Wymagania i narzędzia

3.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają zachowania oraz funkcje, jakie system powinien realizować. Na podstawie analizy problemu oraz specyficznych potrzeb użytkowników wyodrębniono następujący zbiór wymagań:

- rejestracja oraz logowanie użytkowników,
- przeglądanie katalogu produktów sklepu komputerowego,
- dodawanie wybranych komponentów do kreatora zestawu PC,
- automatyczna weryfikacja kompatybilności podzespołów, obejmująca między innymi:
 - zgodność gniazda procesora,
 - typ pamięci RAM,
 - kompatybilność częstotliwości i pojemności pamięci,
 - TDP procesora oraz wymagania energetyczne,
 - dopasowanie komponentów do obudowy,
 - kompatybilność układów chłodzenia.
- wykorzystanie logiki rozmytej do klasyfikacji zestawów pod konkretne kategorie (np. budżetowy, wydajny, energooszczędny),
- generowanie gotowych konfiguracji na podstawie wybranych komponentów,
- dodawanie produktów oraz pełnych zestawów do koszyka,
- modyfikacja zawartości koszyka oraz finalizacja zamówienia.

3.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne opisują właściwości jakościowe systemu, które wpływają na komfort użytkowania, bezpieczeństwo oraz niezawodność.

- **Wydażność:** czas odpowiedzi API nie powinien przekraczać 1 sekundy przy standardowym obciążeniu.
- **Skalowalność:** system powinien umożliwiać uruchomienie w środowisku chmurowym Microsoft Azure.
- **Bezpieczeństwo:** hasła użytkowników muszą być przechowywane w formie zaszyfrowanej (bcrypt).
- **Niezawodność:** system powinien obsługiwać błędy kompatybilności w sposób kontrolowany i komunikatywny.
- **Użyteczność:** interfejs użytkownika powinien być przejrzysty, responsywny i intuicyjny.
- **Przenośność:** frontend uruchamiany w dowolnej nowoczesnej przeglądarce; backend kompatybilny z systemami Linux i Windows.
- **Wysoka dostępność:** system powinien być dostępny w sposób ciągły i niesustanny.

3.3 Przypadki użycia

Najważniejsze przypadki użycia systemu obejmują:

- „Użytkownik rejestruje nowe konto”
- „Użytkownik loguje się do systemu”
- „Użytkownik przegląda dostępne komponenty”
- „Użytkownik buduje własny zestaw PC”
- „System sprawdza kompatybilność elementów”
- „Użytkownik dodaje zestaw do koszyka”
- „Użytkownik realizuje zamówienie”

3.4 Narzędzia i technologie

Realizacja systemu wymagała zastosowania nowoczesnych technologii backendowych, frontendowych oraz narzędzi modelowania. Wszystkie przedstawiono poniżej.

3.4.1 Frontend

- **React** - biblioteka wykorzystywana do tworzenia interaktywnych interfejsów użytkownika.
- **TypeScript** - rozszerzenie JavaScript umożliwiające statyczne typowanie.
- **Vite** - bundler odpowiedzialny za budowanie i szybkie uruchamianie aplikacji.
- **Tailwind CSS** - framework ułatwiający projektowanie responsywnego interfejsu.
- **Node.js** - środowisko wykonawcze dla narzędzi JavaScript.
- **npm** - menedżer pakietów odpowiedzialny za instalację zależności.
- **ESLint** - narzędzie do analizy i kontroli jakości kodu TypeScript i JavaScript.

3.4.2 Backend

- **Python 3** - język implementacji warstwy serwerowej.
- **FastAPI** - framework umożliwiający tworzenie wydajnego REST API.
- **SQLAlchemy** - ORM realizujący komunikację z bazą danych.
- **PostgreSQL** - relacyjna baza danych przechowująca informacje o produktach i zamówieniach.
- **psycopg2** - sterownik PostgreSQL wykorzystywany do komunikacji z bazą.
- **Pydantic** - walidacja modeli danych wykorzystywana przez FastAPI.
- **Uvicorn** - serwer ASGI obsługujący zapytania HTTP.

3.4.3 Modelowanie i narzędzia wspomagające

- **Oracle SQL Developer Data Modeler** - narzędzie użyte do projektowania modelu ERD bazy danych.
- **Git** - system kontroli wersji.
- **GitHub** - repozytorium kodu oraz platforma do zarządzania wersjami projektu.
- **Microsoft Azure** - środowisko do wdrożenia i hostowania bazy danych.

3.5 Metodyka pracy

Prace nad projektem prowadzono w sposób iteracyjny, bazując na podejściu zbliżonym do metodyki Agile. Każdy cykl rozwoju obejmował:

- analizę oraz doprecyzowanie wymagań pod standardy rynkowe,
- projekt danej części funkcjonalnych systemu,
- implementację odpowiednio nowych modułów backendu i frontendu,
- testowanie, w przeznaczonym do tego środowisku, poprawności działania,
- refaktoryzację oraz optymalizację kodu.

Takie podejście umożliwiło równoległe rozwijanie bazy danych, logiki serwera i interfejsu użytkownika, a także szybkie reagowanie na pojawiające się problemy i dostosowywanie projektu do bieżących potrzeb oraz wyzwań.

Rozdział 4

Specyfikacja wewnętrzna

4.1 Przedstawienie idei systemu

Aplikacja powstała jako odpowiedź na nieustannie poszerzający się rynek podzespołów komputerowych, pełniąc funkcję inteligentnego asystenta przy kompletowaniu osobistego zestawu PC. Implementacja logiki rozmytej miała na celu ułatwienie nawet początkującym użytkownikom skuteczne przejście przez ten proces, uwzględniając ich potrzeby takie jak wydajność, energooszczędność czy odpowiedni koszt. System łączy intuicyjny interfejs, znany z innych serwisów internetowych, z zaawansowanymi mechanizmami wspomagającymi, zaprojektowanymi konkretnie pod potrzeby klientów.

4.1.1 Cel powstania aplikacji

Celem powstania aplikacji było stworzenie i następne wdrożenie systemu webowego, mającego za zadanie uproszczenie samodzielnego składania zestawów komputerowych poprzez inteligentną asekurację podczas tego procesu. Jego celem jest eliminacja jak potencjalnie jak największej ilości błędów po stronie użytkowników.

Aplikację projektowano z myślą o:

- intuicyjnym wyborze komponentów z katalogu,
- automatycznej weryfikacji kompatybilności,
- wsparciu w dopasowaniu sprzętu do potrzeb, używając logiki rozmytej,
- możliwości zapisaniu kompatybilnego zestawu oraz o jego zamówieniu.

4.1.2 Innowacyjność systemu

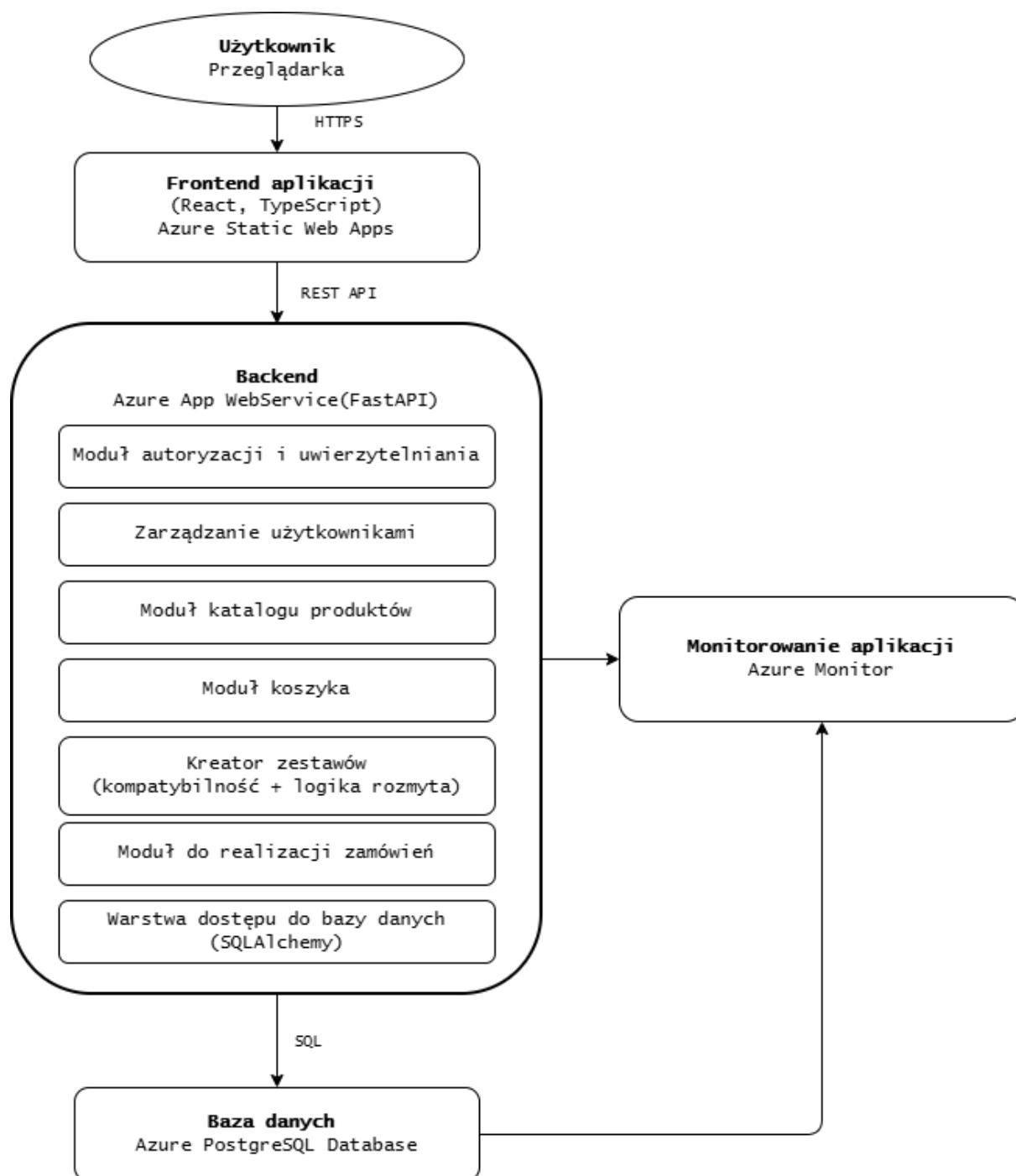
Aplikacja powstała jako inteligentny kreator zestawów komputerowych, składający się z mechanizmów weryfikacji oraz wspomagania decyzji, podczas gdy podobnych rozwiązań nie sposób znaleźć w najpopularniejszych polskich sklepach komputerowych.

Innowacyjność aplikacji wynika z:

- automatycznej weryfikacji kompatybilności, obejmującej wszystkie powszechne znane parametry, takie jak:
 - socket procesora i płyty głównej,
 - typ pamięci RAM,
 - format obudowy,
 - maksymalna suma obciążenia dla konkretnego zasilania,
 - fizyczne wymiary elementów,
 - kompatybilność portów, gniazd oraz złącz.
- zastosowania logiki rozmytej, umożliwiającej dobranie odpowiedniego zestawu pod indywidualne potrzeby, dzieląc je na:
 - budżet,
 - wydajność,
 - energooszczędność,
 - przeznaczenie.
- zapewnionego nieustannego monitorowania stanu aplikacji, dbając o wysoką dostępność serwisu i jego wydajność,
- potencjału rozbudowywania aplikacji w rzeczywistym środowisku produkcyjnym, poprzez dbanie o skalowalność i integralność z najnowszymi technologiami.

4.2 Architektura systemu

4.2.1 Schemat ideowy



Rysunek 4.1: Schemat działania aplikacji

4.2.2 Frontend

Frontend aplikacji pełni ważną funkcję w funkcjonowaniu każdego systemu, ponieważ odpowiada za prezentację danych dostarczonych z backendu. Projekt wykorzystuje nowoczesne technologie React, TypeScript oraz Vite, zapewniając innowacyjność, wydajność oraz skalowalność. Inteligentny kreator zestawów komputerowych to jednostronicowy typ aplikacji webowej, dzięki czemu przełączanie widoku następuje dynamicznie bez konieczności ponownego ładowania zasobów. Takie rozwiązanie gwarantuje wysoką wydajność, prostotę oraz komfort użytkowania.

Zadania realizowane przez interfejs:

- prezentacja katalogu produktów, pobieranych dynamicznie z backendu,
- wybranie komponentów do przeniesienia z katalogu do kreatora,
- obsługa koszyka, dodawanie, modyfikowanie i usuwanie elementów,
- reakcja na działania użytkownika wykorzystując React Hooks,
- wyświetlanie aktualnego stanu kompatybilności zestawów,
- wykonywanie zapytań REST API do backendu w celu weryfikacji danych i aktualizacji stanu aplikacji.

Za wygląd zgodny ze standardami rynkowymi odpowiedzialny jest Tailwind CSS, gwarantując responwność i estetykę rozwiązania. Framework umożliwia konfigurację i nadawanie stylów bezpośrednio w komponentach poprzez gotowe klasy utility. Takie rozwiązanie zapewnia intuicyjne i dynamiczne UI. Za komunikację z backendem odpowiada REST API poprzez zapytania realizowane biblioteką fetch API. Każda interakcja ze stroną powoduje wywołanie właściwego endpoint'u, co dynamicznie aktualizuje stan aplikacji, w tym bazę danych. Projekt wdrożony został w chmurze Microsoft Azure, jako Azure Static Web App, pełniącej rolę hostingu. Takie rozwiązanie umożliwia automatyzację CI/CD, co przekłada się na bezproblemowość i stabilność. Całość stacku technologicznego zapewnia wysoką dostępność aplikacji oraz łatwość korzystania i zarządzania.

Projekt frontendu powstał z myślą o:

- przejrzystości interfejsu,
- prostocie korzystania,
- szybkości działania.

Takie podejście powoduje, że aplikacja jest łatwa w utrzymaniu, skalowalna, wysoko dostępna oraz przejrzysta zarówno dla początkujących jak i zaawansowanych użytkowników.

4.2.3 Backend

Backend, jako jeden z filarów każdej aplikacji, odpowiada za logikę projektu, przetwarzanie danych oraz za kluczową komunikację kodu z bazą danych. Implementacja w języku Python, z wykorzystaniem frameworka FastAPI zapewnia nowoczesność i wydajność usług sieciowych, gwarantując przejrzystość, skalowalność oraz integralność z frontendem. Backend pełni funkcję warstwy pośredniczącej pomiędzy interfejsem a bazą danych, gdzie żądania od strony użytkownika są nieustannie przetwarzane, a następnie przekazywane do odpowiednich modułów. Zwracana odpowiedź, w formacie JSON, umożliwia płynną komunikację w architekturze aplikacji.

Struktura omawianej warstwy zawiera następujące moduły funkcjonalne:

- moduł autoryzacji i uwierzytelniania - odpowiada za rejestrację, logowanie i odpowiednie weryfikowanie danych użytkownika. Wykorzystuje algorytm bcrypt do zapewnienia bezpieczeństwa haseł,
- moduł zarządzania użytkownikami - umożliwia monitorowanie historii zamówień,
- moduł katalogu produktów - umożliwia przeglądanie oferty sklepu, pobieranie listy części, filtrowanie i sortowanie,
- moduł kreatora - zapewnia możliwość zbierania danych o wybranych komponentach, gromadząc je w strukturze lokalnej sesji użytkownika, pozwalając na transport informacji z oferty do kreatora,
- moduł weryfikacji kompatybilności - odpowiada za porównywanie parametrów elementów zestawu, analizując zgodność pomiędzy nimi, stanowiąc rdzeń i główną funkcjonalność projektu,
- moduł logiki rozmytej - odpowiedzialny jest za klasyfikację zestawu do jednej z kategorii,
- moduł koszyka i zamówień - zadaniem tego modułu jest dodawanie elementów do koszyka, nieustanne monitorowanie jego stanu i modyfikowanie zawartości.

Komunikacja pomiędzy backendem a bazą danych zaprojektowana została przy pomocy SQLAlchemy, który pełni funkcję warstwy ORM. Takie rozwiązanie zapewnia odwzorowanie struktury tabel w postaci obiektów, znacząco ułatwiając implementację logiki, minimalizując ryzyko błędów oraz zwiększając bezpieczeństwo. Operacje wykonywane w ten sposób określamy jako bezpieczne i atomowe. Wdrożenie w postaci Azure App Service, zapewnia skalowalność, prostotę utrzymania oraz wysoką dostępność, co jest niezwykle istotne podczas projektowania aplikacji przeznaczonej dla klientów. Uruchamianie nowych wersji odbywa się automatycznie poprzez mechanizm stałej implementacji i stałego dostarczania, znanego szerzej jako CI/CD.

Backend zaprojektowany został z myślą o:

- wysokiej wydajności,
- przejrzystości kodu,
- bezpieczeństwie danych,
- skalowalności,
- niezawodności,
- wysokiej dostępności.

Realizacja tej części projektu w sposób opisany powyżej stanowi stabilną i skalowalną podstawę działania aplikacji, gwarantując poprawne komunikowanie się żądań użytkownika z bazą danych.

4.2.4 Baza danych

Baza danych jest fundamentem każdej aplikacji biznesowej, gdyż odpowiedzialna jest za trwałe przechowywanie informacji o użytkownikach, produktach oraz zamówieniach. Zaprojektowana została w oparciu o zasady normalizacji oraz analizę charakterystycznych właściwości komponentów, zapewniając integralność danych oraz efektywne weryfikowanie kompatybilności elementów.

Model i schemat danych zaprojektowany został przy użyciu Oracle SQL Developer Data Modeler, co umożliwiło utworzenie diagramu ERD oraz automatyczne wygenerowanie definicji tabel. Wyeksportowana baza jako plik DDL wdrożona została do środowiska DataGrip, w którym wypełniono ją przygotowanymi INSERT'ami, zaciągniętymi z publicznego repoztorium komponentów[3], w celu implementacji danych produktów. System operował w oparciu o PostgreSQL, pełniący funkcję środowiska testowego wykorzystywanego do sprawnego implementowania backendu i modułów logiki. Gdy projekt przeszedł do etapu wdrożenia produkcyjnego, baza danych została zmigrowana do chmury Microsoft Azure, korzystając z gotowych poleceń podanych w dokumentacji Microsoft Azure[1]. Przeniesiona struktura zaimplementowana została jako Azure PostgreSQL Database, co dzięki funkcjonalności grupowania zasobów Azure zapewnia integrację z produkcyjną wersją backendu uruchomioną w Azure App Service.

Baza Danych spełnia wymagi trzeciej postaci normalnej (3NF), co oznacza, że nie zawiera redundancji danych, wszystkie atrybuty niekluczowe są w pełni zależne od klucza głównego właściwych tabel. Właściwość ta pozwala na:

- zminimalizowanie ryzyka niespójnych danych,
- zwiększenie wydajności zapytań,
- eliminację duplikatów.

Kluczową rolę w bazie danych pełni tabela Produkty, będąc ogólnym rejestrem wszystkich komponentów dostępnych w ofercie. Przechowuje ona podstawowe informacje o produktach, natomiast szczegółowe parametry znajdują się w dedykowanych tabelach zależnych od typu podzespołu. Takie rozwiązanie eliminuje redundację danych oraz dodaje przejrzystości do przechowywania właściwości.

W celu przechowywania wielu komponentów w jednym zestawie, koszyku lub zamówieniu zastosowano liczne relacje typu M:N, które zostały zrealizowane przy pomocy tabel łączących. Całość struktury została zabezpieczona kluczami obcymi, co zapewnia pełną integralność referencyjną oraz uniemożliwia tworzenie błędnych powiązań między danymi. Modularny charakter modelu umożliwia jego dalszy rozwój, w tym dodawanie nowych typów komponentów.

Dzięki zastosowaniu odpowiednich narzędzi, normalizacji oraz chmurowej infrastruktury Azure, baza danych stanowi stabilny i skalowalny fundament systemu, wspierający zarówno logikę kompatybilności, jak i zarządzanie koszykami, zamówieniami oraz zestawami użytkowników.

4.2.5 Infrastruktura chmurowa

Wybór infrastruktury chmurowej jest podstawowym aspektem każdego nastawionego na wysoką dostępność projektu. Chmura Microsoft Azure gwarantuje skalowalność i bezpieczne środowisko, umożliwiając integrację z poszczególnymi komponentami systemu oraz automatyzację wdrażania kolejnych części aplikacji.

Organizację pracy znacząco upraszcza ulokowanie środowiska produkcyjnego w oparciu o grupę zasobów Azure, która zbiera całość w jednej strukturze administracyjnej. Takie rozwiązanie gwarantuje intuicyjne i kontrolowane zarządzanie usługami, co pozwala nieprzerwanie monitorować zużycie zasobów oraz koszty serwisu.

Projekt operował na kredytach Azure o równowartości 100 USD, przyznanych przez Microsoft w ramach programu Azure for Students, pozwalając na wdrożenie aplikacji bez generowania dodatkowych kosztów. Środki te umożliwiły uruchomienie wszystkich wymaganych usług, takich jak:

- Azure App Service,
- Azure Static Web App,
- Azure Database for PostgreSQL.

Azure Static Web App

Warstwa frontend'u zaimplementowana została w usłudze Static Web Apps, co umożliwiło prezentowanie strony w sieci Internet. Usługa ta obsługuje:

- automatyzację CI/CD,
- integrację z backendem.

Stanowi to lekkie, skalowalne rozwiązanie, zapewniając obsługę wielu użytkowników jednocześnie.

Azure App Service

Warstwa backend'u została wdrożona przy użyciu usługi App Service. Oparty na frameworku FastAPI i języku Python osadzona została jako aplikacja, działając na serwerze automatyzowanym przez usługi Azure, zapewniając dzięki temu:

- uproszczone wdrażanie aktualizacji,
- zarządzanie środowiskiem wykonawczym,
- integrację z usługami monitorującymi Azure,
- stabilność,
- wysoką dostępność.

Takie rozwiązanie uodparnia backend na obciążenia i znacząco redukuje wkład użytkownika w administrację w czasie rzeczywistym.

Azure Database for PostgreSQL

Dane o podzespołach przetrzymywane są w usłudze Azure Database for PostgreSQL, zapewniając środowisko do zarządzania relacyjnymi bazami danych. Wdrożenie polegało na zmigrowaniu bazy PostgreSQL do Azure [1], obejmując elementy takie jak:

- eksport struktury i danych,
- dostosowanie typów danych,
- weryfikację integralności.

Rozwiązanie to zapewnia dostęp do bazy z dowolnego urządzenia, gwarantując elastyczność pracy i wysoką niezawodność. Przechowywanie danych w chmurze daje możliwość elastycznego dostępu do danych zależnie od potrzeb, redukując koszty utrzymania.

Azure Monitor

Aplikacja jest monitorowana w czasie rzeczywistym za pomocą usług:

- Azure Monitor,
- Application Insights.

Takie rozwiązanie umożliwia nieustanne wykrywanie problemów, reagowanie na zwiększony ruch oraz automatyzację dopasowywania zasobów do aktualnych potrzeb.

Koszty i optymalizacja

W trakcie wdrażania przeanalizowany został potencjalny koszt utrzymania aplikacji, dobierając odpowiednie rozwiązania, o możliwie niskich parametrach z uwagi na niski poziom ruchu na stronie. Wprowadzone modyfikacje nie mają wpływu na funkcjonalność, pozwalają one na oszczędność środków, których zużycie nawet na średnich komponentach przekracza założony budżet. Wybrane do tego zostały następujące rozwiązania:

- wybranie maszyny wirtualnej o najniższych możliwych parametrach,
- ustawienie automatycznego usypiania systemu w przypadku ruchu poniżej progu zdefiniowanego ręcznie,
- ograniczenie ruchu sieciowego do testowej podsiatki wirtualnej.

W ten sposób możliwe stało się zmieszczenie w oferowanych przez Microsoft kredytach, ale również przygotowało projekt do ewentualnych przyszłych modyfikacji.

Architektura logiczna

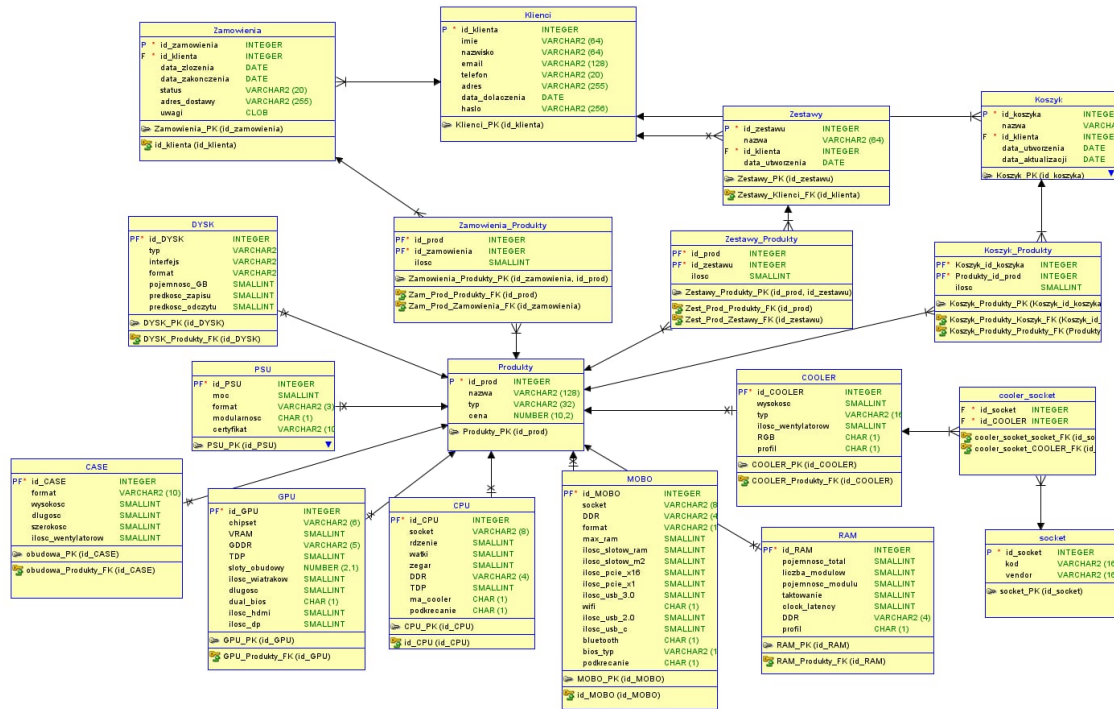
Infrastruktura została umieszczona w jednej grupie zasobów, umożliwiając spójne zarządzanie, wzajemny dostęp do zasobów oraz przystępną modyfikację pojedynczych modułów. Działanie infrastruktury przebiega w następujący sposób:

- połączenie użytkownika z frontendem (Static Web App),
- komunikacja na linii frontend - backend (RestAPI),
- komunikacja na linii backend - Baza PostgreSQL,
- monitorowanie komponentów, zwracanie dedykowanych logów, które analizowane przez Azure Monitor dostosowują parametry aplikacji.

Taki sposób implementacji zapewnia wysoką wydajność, stabilność, skalowalność oraz integralność, czyniąc aplikację gotową do działania w produkcyjnym środowisku.

4.3 Baza danych

4.3.1 Model danych - ERD



Rysunek 4.2: Diagram ERD bazy danych

Na rysunku przedstawiono diagram ERD bazy danych systemu sklepu komputerowego z kreatorem zestawów PC. Model odzwierciedla główne obszary funkcjonalne: aplikacji

- zarządzanie klientami,
- zarządzanie produktami,
- zarządzanie koszykami,
- zarządzanie zamówieniami.

Centralne miejsce w diagramie zajmuje tabela *Produkty*, pełniąca rolę nadrzędnego rejestru wszystkich pozycji dostępnych w systemie. Zawiera ona podstawowe informacje o każdym produkcie, takie jak nazwa, typ oraz cena, natomiast szczegółowe parametry techniczne przechowywane są w dedykowanych tabelach odpowiadających konkretnym kategoriom sprzętu: CPU, GPU, RAM, MOBO, PSU, COOLER, DYSK oraz CASE. Każda z tych tabel posiada klucz obcy wskazujący na rekord w tabeli *Produkty*, co pozwala uniknąć redundancji danych oraz ułatwia zarządzanie wspólnymi atrybutami produktów.

Relacje związane z procesem zakupowym odwzorowane są poprzez tabele Klienci, Koszyk, Zamówienia oraz Zestawy. Każdy klient może posiadać wiele koszyków, zamówień oraz zapisanych zestawów, co odzwierciedlono relacjami 1:N. Zawartość koszyka, zamówienia oraz zestawu została zrealizowana za pomocą tabel łączących Koszyk_Produkty, Zamówienia_Produkty oraz Zestawy_Produkty, które przechowują pary identyfikatorów produktu i obiektu nadrzędnego wraz z informacją o liczbie sztuk. Umożliwia to poprawne odwzorowanie relacji wiele-do-wielu pomiędzy produktami a koszykami, zamówieniami i zestawami.

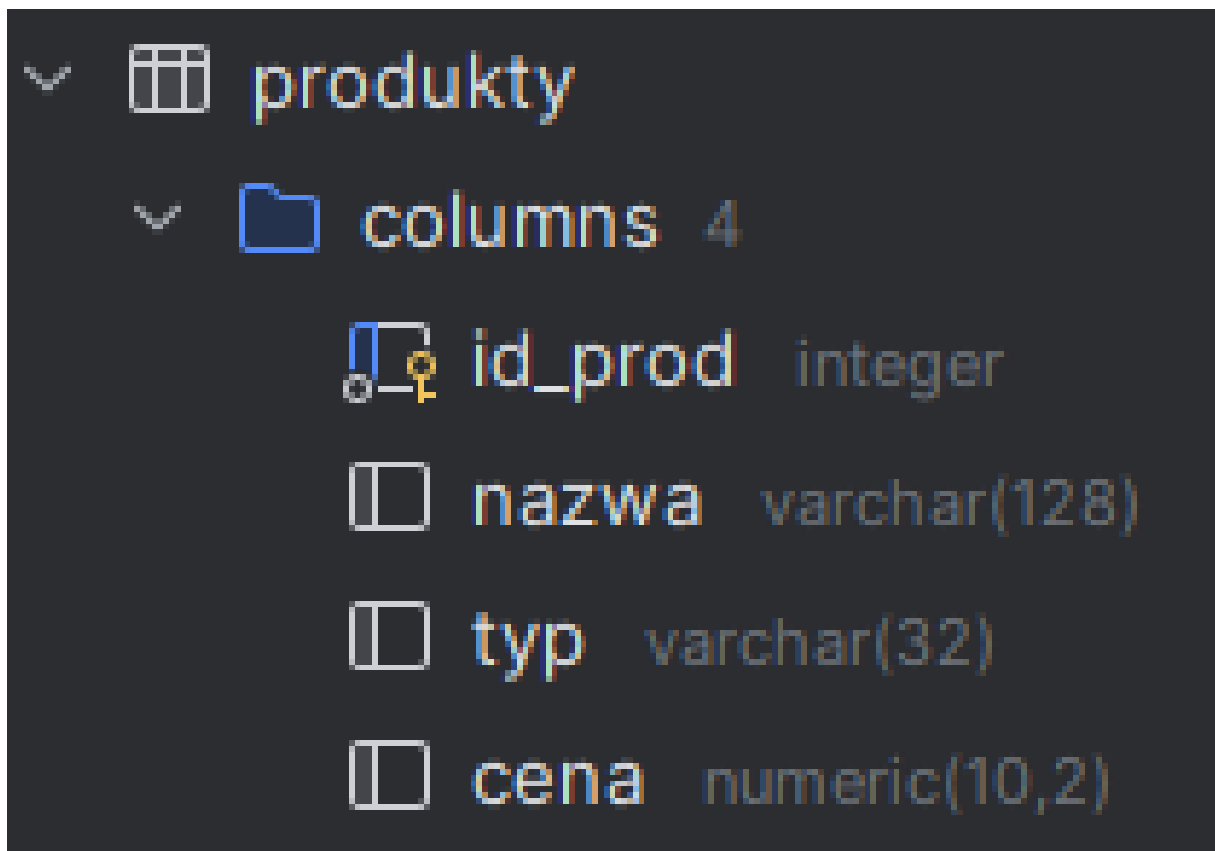
Istotnym elementem modelu są tabele socket oraz cooler_socket, pełniące rolę słowników technicznych wykorzystywanych w logice kompatybilności. Tabela socket przechowuje informacje o typach gniazd procesorów, natomiast tabela cooler_socket łączy coolery z obsługiwanymi przez nie socketami. Rozwiązanie to umożliwia powiązanie procesorów, płyt głównych oraz systemów chłodzenia oraz ułatwia implementację reguł sprawdzających zgodność komponentów.

Zaprojektowany model danych zachowuje spójność referencyjną dzięki zastosowaniu kluczy głównych i obcych, a rozdzielenie danych ogólnych i szczegółowych na osobne tabele umożliwia utrzymanie bazy w co najmniej trzeciej postaci normalnej. Takie podejście ułatwia dalszą rozbudowę systemu o nowe typy podzespołów oraz dodatkowe funkcje biznesowe, jednocześnie zapewniając integralność i wysoką jakość przechowywanych danych.

4.3.2 Opis najważniejszych encji

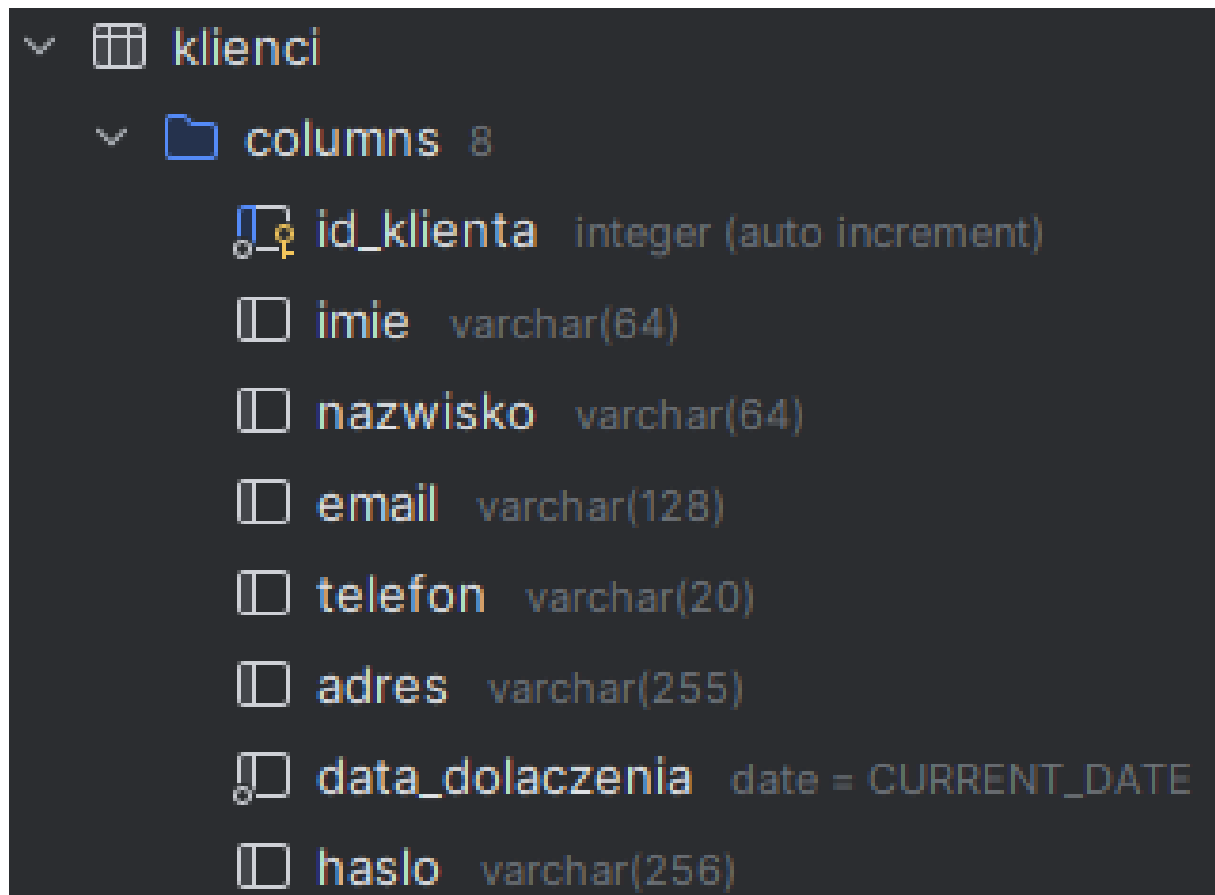
Tabela produkty

Najbardziej istotną, pełniącą funkcję głównej, tabelą w całej bazie danych jest tabela produkty. Kluczem tabeli jest `id_prod`, klucz ten podstawą działania aplikacji. Encja produkty zawiera również pełną nazwę produktu w typie `varchar`, typ produktu, który umożliwia prowadzenie poprawnych operacji w kreatorze zestawów, stanowiąc pole odpowiedzialne za łączenie odpowiednich tabel z dodatkowymi danymi konkretnego produktu, zależnie od jego rodzaju. Tabela zawiera również kolumnę `cena` w typie `numeric`, który wybrano w celu zachowania spójności działania programu.



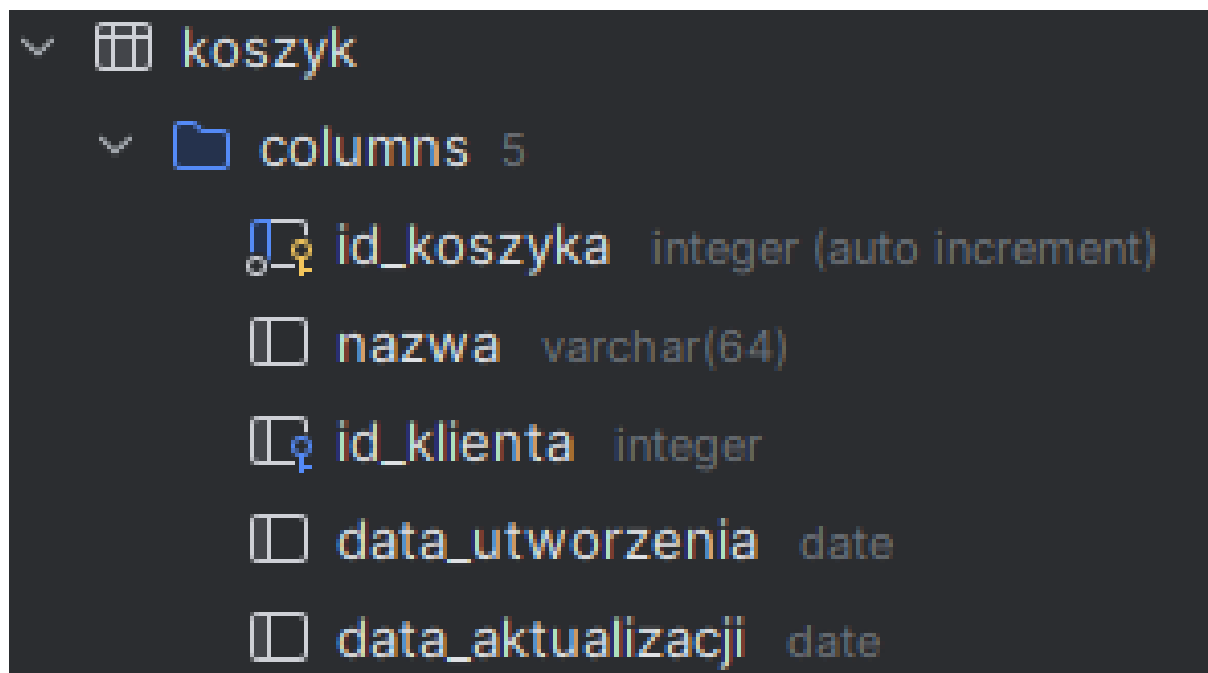
Rysunek 4.3: Tabela Produkty w programie DataGrip

Tabela Klienci



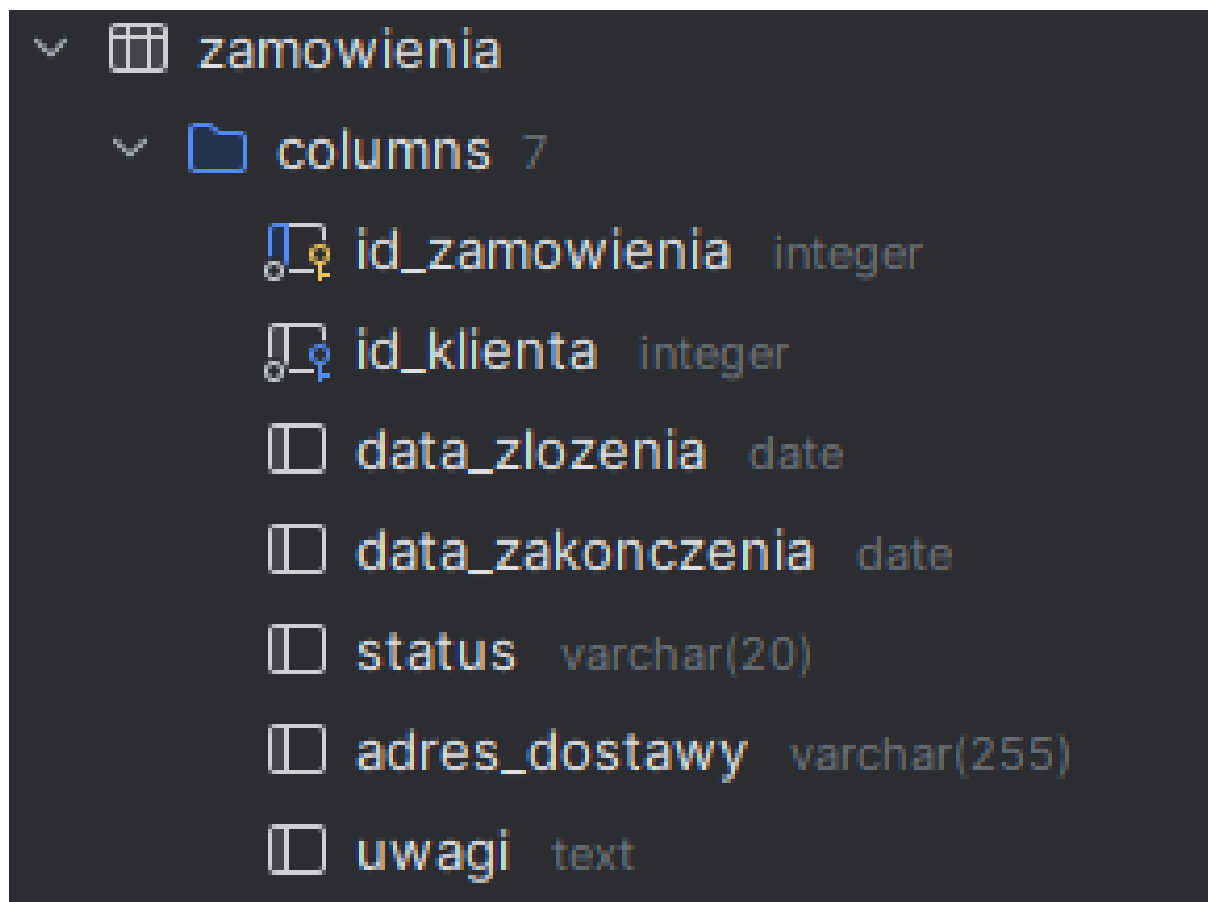
Rysunek 4.4: Tabela Klienci w programie DataGrip

Tabela Koszyk



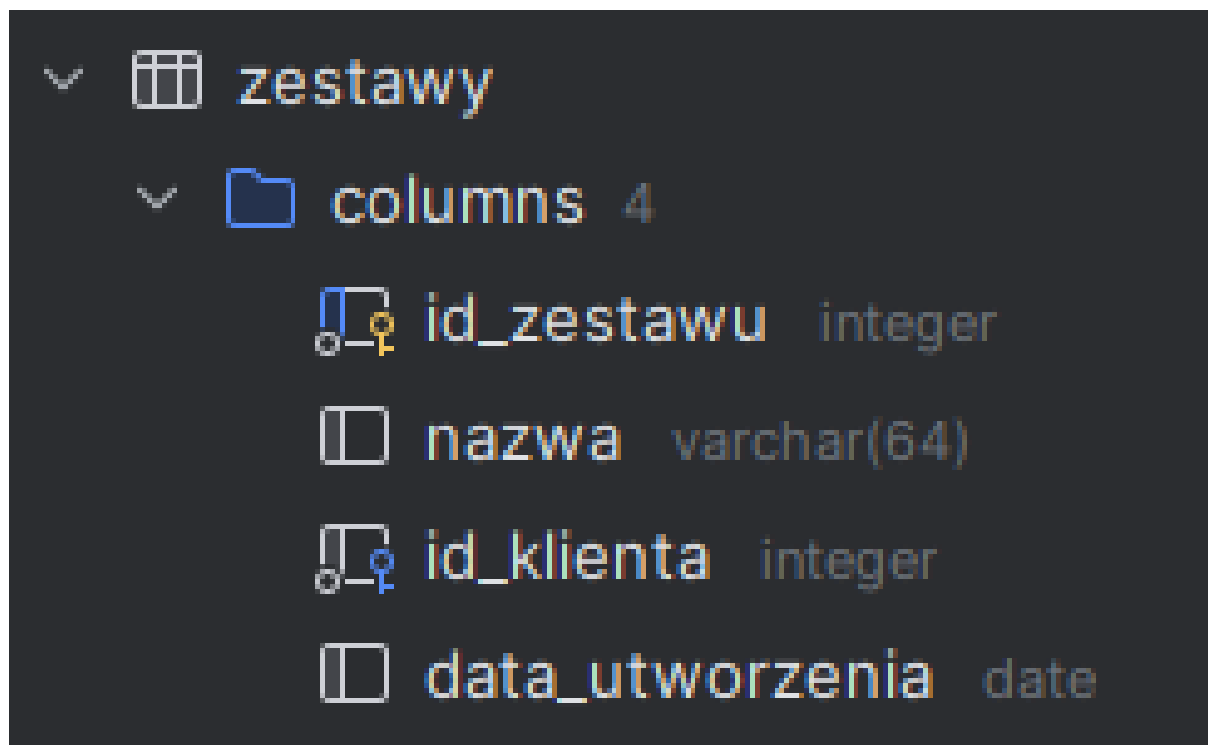
Rysunek 4.5: Tabela Koszyk w programie DataGrip

Tabela Zamówienia



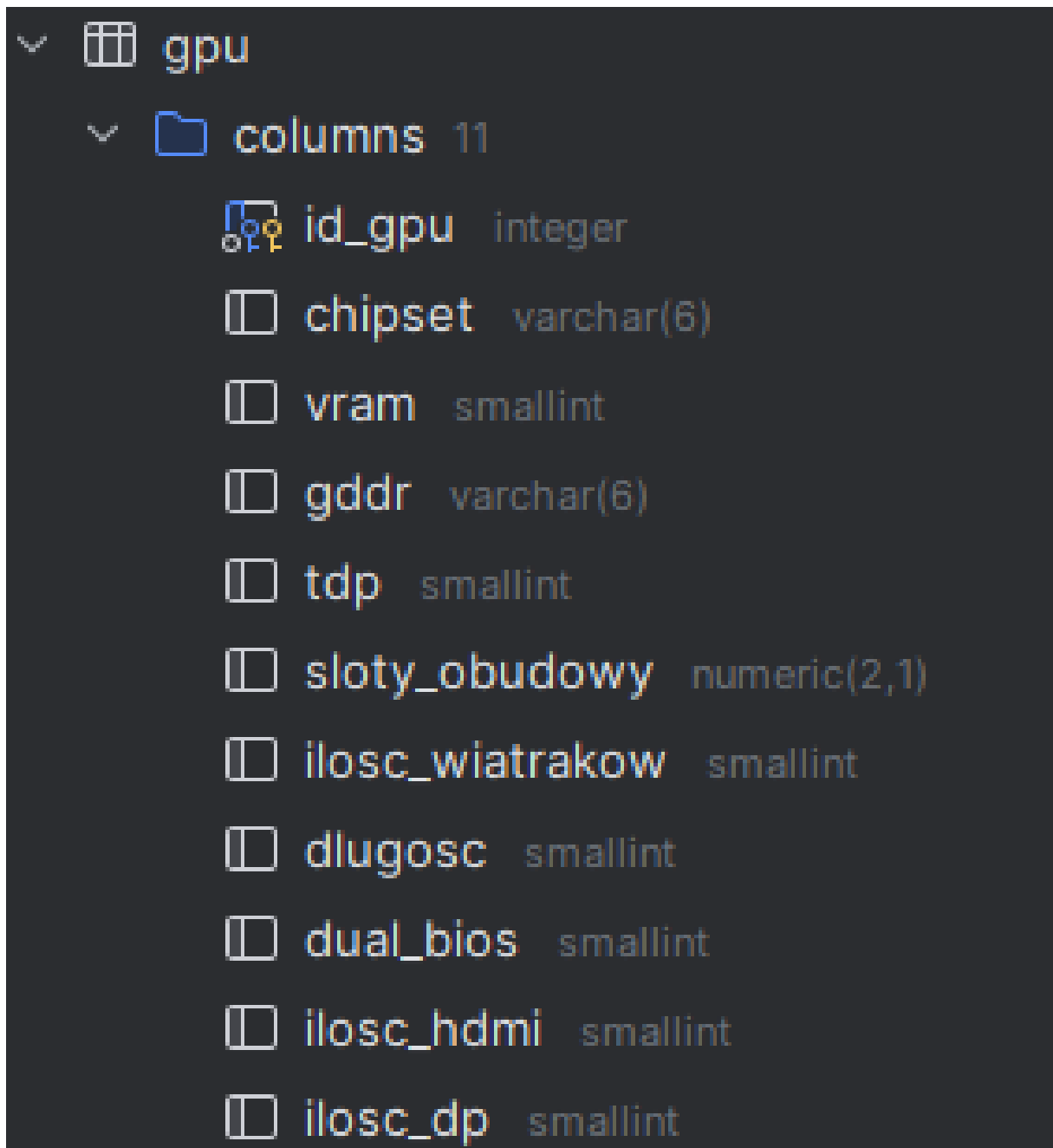
Rysunek 4.6: Tabela Zamówienia w programie DataGrip

Tabela Zestawy



Rysunek 4.7: Tabela Zestawy w programie DataGrip

Tabela GPU



▼	gpu
▼	columns 11
id_gpu	integer
chipset	varchar(6)
vram	smallint
gddr	varchar(6)
tdp	smallint
sloty_obudowy	numeric(2,1)
ilosc_wiatrakow	smallint
dlugosc	smallint
dual_bios	smallint
ilosc_hdmi	smallint
ilosc_dp	smallint

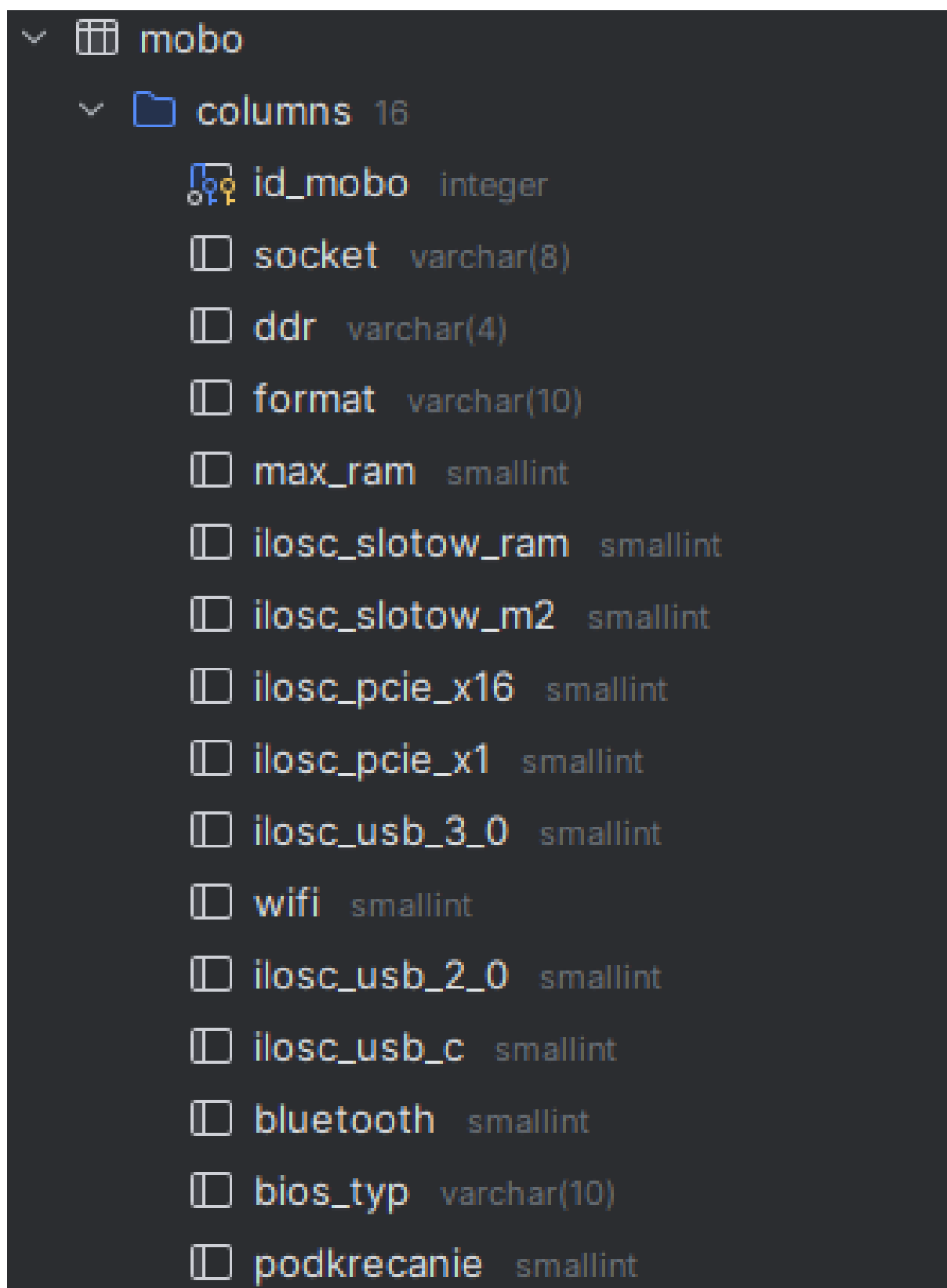
Rysunek 4.8: Tabela GPU w programie DataGrip

Tabela CPU

cpu
columns 9
id_cpu integer
socket varchar(8)
rdzenie smallint
watki smallint
zegar smallint
tdp smallint
ma_cooler smallint
podkrecanie smallint
ma_integre smallint

Rysunek 4.9: Tabela CPU w programie DataGrip

Tabela MOBO



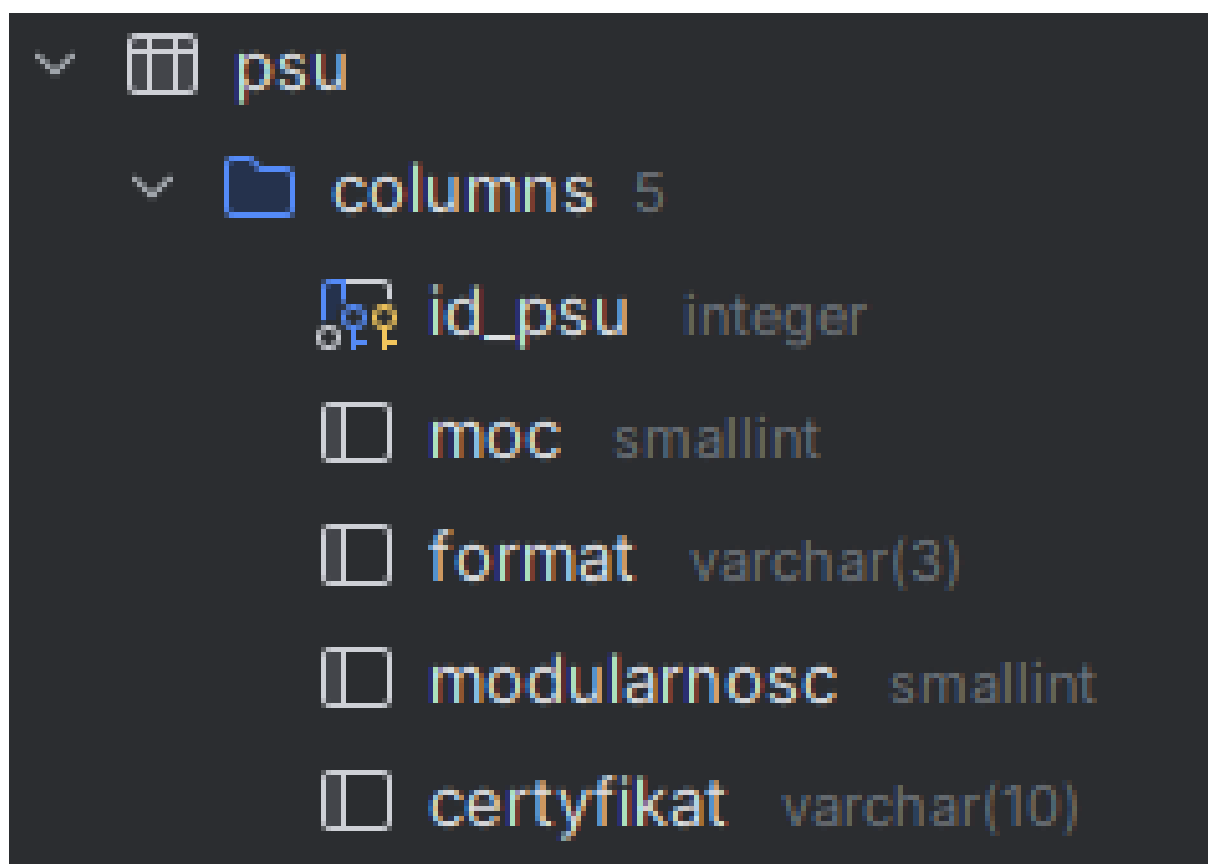
Rysunek 4.10: Tabela MOBO w programie DataGrip

Tabela RAM

▼	📊	ram
▼	📁	columns 8
	🔑	id_ram integer
	📦	pojemnosc_total smallint
	📦	liczba_modulow smallint
	📦	pojemnosc_modulu smallint
	📦	taktowanie smallint
	📦	clock_latency smallint
	📦	ddr varchar(4)
	📦	profil smallint

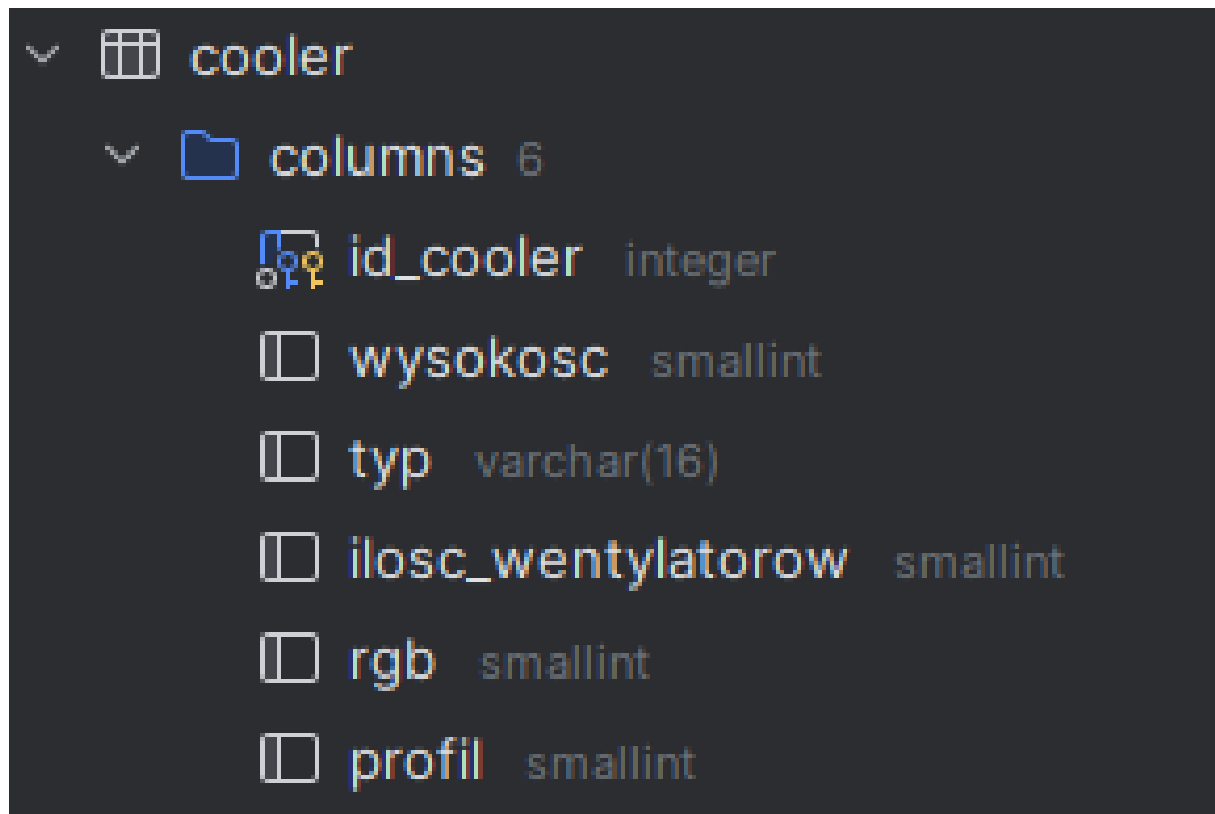
Rysunek 4.11: Tabela RAM w programie DataGrip

Tabela PSU



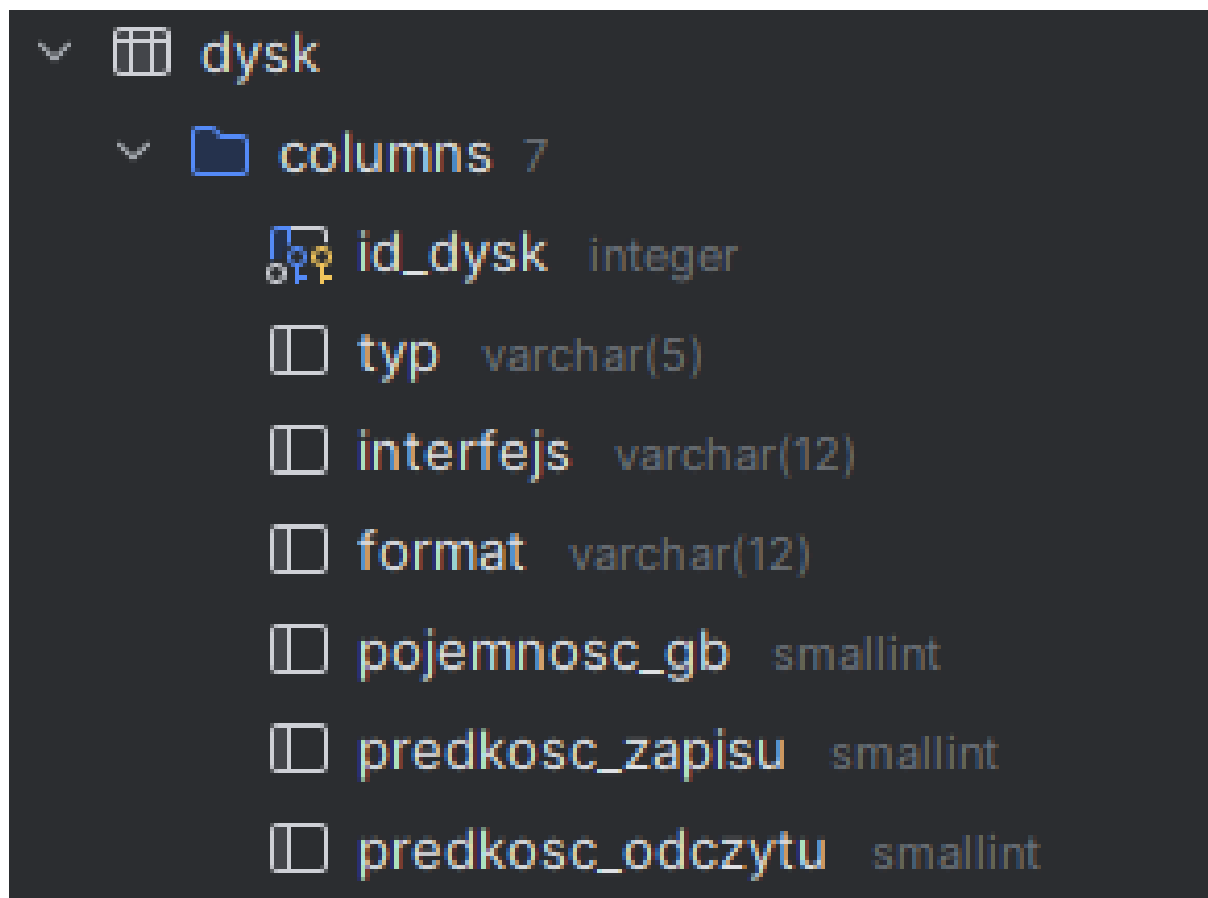
Rysunek 4.12: Tabela PSU w programie DataGrip

Tabela Cooler



Rysunek 4.13: Tabela Cooler w programie DataGrip

Tabela Dysk



Rysunek 4.14: Tabela Dysk w programie DataGrip

4.4 Komponenty i moduły systemu

4.4.1 Moduł kompatybilności podzespołów

4.4.2 Moduł logiki rozmytej

4.4.3 Moduł zarządzania koszykiem i zamówieniami

4.5 Najważniejsze algorytmy

4.6 Diagramy UML

4.7 Szczegóły implementacji wybranych elementów

Rozdział 5

Weryfikacja i walidacja

5.1 Cel weryfikacji systemu

5.2 Metody testowania

5.3 Organizacja eksperymentów

5.4 Przypadki testowe

5.5 Wyniki testów

Tabela 5.1: Przykładowe wyniki testów kompatybilności zestawów komputerowych.

ID	Test	Dane wejściowe	Problem	Wynik
1	CPU + płyta główna	Ryzen 5 7600 + B650	Brak	Kompatybilne
2	CPU + płyta główna	i5-12600K + B550	Różne sockety	Niekompatybilne
3	RAM + płyta główna	DDR5 6000 + B460	RAM DDR5 + MOBO DDR4	Niekompatybilne
4	GPU + obudowa	RTX 4070 (330 mm) + obudowa 300 mm	Zbyt długie GPU	Niekompatybilne
5	Zasilacz + CPU/GPU	PSU 400W + RTX 3080	Zbyt mała moc	Niekompatybilne
6	Chłodzenie + socket	Cooler AM4 + CPU LGA1700	Niezgodne typy chłodzenia	Niekompatybilne
7	Zestaw kompletny	CPU + GPU + RAM + Mobo + PSU zgodne	Brak	Kompatybilne
8	Zestaw błędny	3 niezgodności (DDR, TDP, GPU)	Złożona	Niekompatybilne

5.6 Wykryte i usunięte błędy

5.7 Ocena poprawności działania systemu

Rozdział 6

Podsumowanie i wnioski

Celem niniejszego rozdziału jest podsumowanie rezultatów uzyskanych podczas realizacji projektu oraz ocena stopnia spełnienia celów pracy. Dodatkowo przedstawiono wnioski dotyczące dalszego możliwego rozwoju systemu oraz trudności napotkane w trakcie implementacji.

6.1 Realizacja celów pracy

6.2 Ocena otrzymanych rezultatów

6.3 Trudności napotkane podczas realizacji

6.4 Możliwe kierunki dalszego rozwoju

6.5 Wnioski końcowe

Bibliografia

- [1] *Migrate PostgreSQL to Azure Database for PostgreSQL using Azure Database Migration Service*. 2025. URL: <https://learn.microsoft.com/en-us/azure/dms/tutorial-postgresql-azure-postgresql-online-portal> (dostęp 12.10.2025).
- [2] *Morele.net - sklep komputerowy*. 2025. URL: <https://www.morele.net/> (dostęp 17.11.2025).
- [3] *PC Part Dataset - Public Hardware Dataset for Computer Components*. 2025. URL: <https://github.com/docyx/pc-part-dataset> (dostęp 11.09.2025).
- [4] *X-kom - sklep komputerowy*. 2025. URL: <https://www.x-kom.pl/> (dostęp 17.11.2025).

Dodatki

Spis skrótów i symboli

- API interfejs programistyczny aplikacji (ang. *Application Programming Interface*)
- ASGI asynchroniczny interfejs serwera aplikacji (ang. *Asynchronous Server Gateway Interface*)
- ATX standard rozmiaru płyt głównych i obudów komputerowych (ang. *Advanced Technology eXtended*)
- CPU procesor centralny (ang. *Central Processing Unit*)
- CRUD podstawowe operacje na danych: tworzenie, odczyt, modyfikacja, usuwanie (ang. *Create, Read, Update, Delete*)
- CSS kaskadowe arkusze stylów (ang. *Cascading Style Sheets*)
- DDR standard pamięci RAM (*Double Data Rate*)
- ERD diagram związków encji (ang. *Entity–Relationship Diagram*)
- GPU procesor graficzny (ang. *Graphics Processing Unit*)
- HTML język znaczników stron WWW (ang. *HyperText Markup Language*)
- HTTP protokół komunikacji klient-serwer (ang. *HyperText Transfer Protocol*)
- JSON format danych tekstowych (ang. *JavaScript Object Notation*)
- ORM mapowanie obiektowo-relacyjne (ang. *Object–Relational Mapping*)
- PSU zasilacz komputerowy (ang. *Power Supply Unit*)
- RAM pamięć operacyjna (ang. *Random Access Memory*)
- REST styl architektury usług sieciowych (ang. *Representational State Transfer*)
- SQL język zapytań do baz danych (ang. *Structured Query Language*)
- SSD dysk półprzewodnikowy (ang. *Solid State Drive*)

TDP maksymalna ilość ciepła odprowadzana przez układ (ang. *Thermal Design Power*)

TS język programowania TypeScript (ang. *TypeScript*)

UI interfejs użytkownika (ang. *User Interface*)

UML zunifikowany język modelowania (ang. *Unified Modeling Language*)

URL adres zasobu internetowego (ang. *Uniform Resource Locator*)

WWW ogólnosiwiatowa sieć internetowa (ang. *World Wide Web*)

Źródła

Jeżeli w pracy konieczne jest umieszczenie długich fragmentów kodu źródłowego, należy je przenieść w to miejsce.

```
1 if (_nClusters < 1)
2     throw std::string ("unknown number of clusters");
3 if (_nIterations < 1 and _epsilon < 0)
4     throw std::string ("You should set a maximal number of
        iteration or minimal difference — epsilon.");
5 if (_nIterations > 0 and _epsilon > 0)
6     throw std::string ("Both number of iterations and minimal
        epsilon set — you should set either number of iterations
        or minimal epsilon.");
```

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- dane testowe,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

4.1	Schemat działania aplikacji	20
4.2	Diagram ERD bazy danych	28
4.3	Tabela Produkty w programie DataGrip	30
4.4	Tabela Klienci w programie DataGrip	31
4.5	Tabela Koszyk w programie DataGrip	32
4.6	Tabela Zamówienia w programie DataGrip	33
4.7	Tabela Zestawy w programie DataGrip	34
4.8	Tabela GPU w programie DataGrip	35
4.9	Tabela CPU w programie DataGrip	36
4.10	Tabela MOBO w programie DataGrip	38
4.11	Tabela RAM w programie DataGrip	39
4.12	Tabela PSU w programie DataGrip	40
4.13	Tabela Cooler w programie DataGrip	41
4.14	Tabela Dysk w programie DataGrip	42

Spis tabel

5.1	Przykładowe wyniki testów kompatybilności zestawów komputerowych. . .	44
-----	---	----